MARKETPLACE KOI PACK v1.2.3 · ACCEPTANCE TESTING

Test Guide

Ten tests for the official pack from demisto/content — Packs/Koi/Integrations/Koi/Koi.yml — pack version 1.2.3, 13 commands, integration only. NOT the custom in-house KOI pack (v1.3.0, 26 commands); both present as "KOI" (test 1 tells them apart).

Optional companion pack: *KOI Content Extension (KoiContentExtension v1.0.0, community support) layers parsing & modeling rules, 10 playbooks and a dashboard on top of this integration — SEPARATE, additional content, not part of this pack and not the custom v1.3.0 pack.*

10	13	8	5	0
tests	commands	non-mutating	state-changing	playbooks in pack

Command surface generated at build time from reference/marketplace-pack.json (upstream demisto/content Packs/Koi/Integrations/Koi/Koi.yml, md5 5497cdddedeb0c0d7d0b371aa075a64c, checked against master 2026-07-20). Tenant observations cited to VERIFIED_FACTS.md, verified 20–21 July 2026.

BEFORE YOU START

Two different packs are both called KOI

They share a pack name, an integration id, a category, an author and the koi-* command prefix. They cannot coexist on one tenant — installing one overwrites the other. Everything in this guide is written for the left-hand column.

What to check	THIS pack — Marketplace	The custom pack — out of scope
Pack version	1.2.3	1.3.0
Commands	13	26
Content items	Integration only	Integration + 10 playbooks + rules + dashboard
Context prefixes	`KOI.Event, Koi.Allowlist, Koi.Blocklist, Koi.Inventory, Koi.Policy`	17 prefixes, incl. Koi.Device.*
Parsing / modeling rules	None ship	Both ship
fromversion	6.10.0	8.2.0 integration / 8.4.0 rules
Docker image	`demisto/fastapi:0.125.0.10158186`	demisto/fastapi:0.125.0.9094740

Preserve the casing inconsistency in both packs: events are KOI.Event., everything else is Koi.*. "Fixing" it breaks DT paths.*

Marketplace KOI pack v1.2.3 · demisto/content Packs/Koi/Integrations/Koi/Koi.yml · 13 commands — NOT the custom in-house pack (v1.3.0, 26 commands)

TEST PLAN

Ten tests, in order

1

Pack identity

currentVersion is 1.2.3 and 13 commands are registered

2

Instance config + Test

The instance saves and the Test button returns Success

3

Policy & governance reads

koi-policy-list, koi-allowlist-get, koi-blocklist-get

4

Inventory reads

list, item-get, item-endpoints-list return Koi.Inventory.*

5

Advanced search

koi-inventory-search with a query-builder filter

6

Event command

koi-get-events, debug only, push disabled

7

The mcp_servers view

Works, but is missing from the argument dropdown

8

State-changing commands

Controlled protocol — record, change, reverse

9

Event collection

koi_koi_raw is populated by the collector

10

Alerts vs Audit, and XDM

Split by source_log_type; XDM is empty by design

Tests 1 and 2 are gates: do not interpret any later result until both pass.
Test 8 is the only one that writes to KOI — run it last, on a disposable item, and reverse it.
Tests 3, 4, 5 and 7 are read-only and safe to repeat. Test 6 is NOT: the pack calls koi-get-events debugging-only — it may duplicate events, exceed rate limits or disrupt the fetch, and should_push_events=true writes events into koi_koi_raw.

PRECONDITIONS

What must be true before test 1

For every test

Identity, access, and a place to type commands

- ✓ The Marketplace KOI pack v1.2.3 installed — and the custom pack NOT installed
- ✓ Cortex platform at or above fromversion 6.10.0
- ✓ A KOI API key, and an egress path KOI accepts (direct tenant egress or an engine)
- ✓ A war room / playground where you can type ! commands
- ✓ Permission to read the instance configuration

For tests 9 and 10 only

Event collection into the dataset — XSIAM / platform only

- ✓ Fetch events enabled — the Collect section is hidden on XSOAR, where the collector is off
- ✓ Fetch event types set — default is Alerts,Audit
- ✓ At least two completed fetch cycles
- ✓ XQL search access on the tenant
- ✓ A KOI scan has run on a monitored host and a real change occurred there — koi_koi_raw moves only then (§7b/§8)

! **Know how your instance reaches KOI — and what this guide has NOT seen**

On the tenant behind this guide, both KOI instances run through a Cortex engine rather than direct tenant egress (VERIFIED_FACTS §2). That matters for every connectivity conclusion: a reachability check from the tenant itself proves nothing about the path the commands actually take. Separately: the koi-* commands were never executed from a war room on that tenant (VERIFIED_FACTS §6), so every war-room table, context path and human-readable output stated in this guide is asserted from the pack YAML, not observed. Running these tests is what closes that gap.

TEST 1 · GATE

Confirm WHICH KOI pack is installed

Steps

1. Settings → Marketplace → Installed content → open the pack named KOI.
2. Read the installed version. It must be 1.2.3.
3. Open Settings → Integrations → Instances, find the KOI integration.
4. Open its command list (or type ! in a war room and filter on koi-).
5. Count the commands. There must be exactly 13.
6. Spot-check that these are ABSENT — they belong to the other pack:

   `koi-devices-list` `koi-koidex-search` `koi-findings-list`

7. Confirm the pack contains no playbooks, no rules and no dashboard.

Expected result

- ✓ Pack currentVersion reads 1.2.3.
- ✓ Exactly 13 koi-* commands are registered.
- ✓ Context prefixes offered are only: KOI.Event, Koi.Allowlist, Koi.Blocklist, Koi.Inventory, Koi.Policy.
- ✓ No Koi.Device.* prefix exists anywhere.
- ✓ The pack lists one content item: the KOI integration.
- ✓ None of the three spot-check commands resolve.

! **Real failure vs empty-but-correct**

This test has no empty-but-correct outcome — it is pass or fail. If you see 26 commands, or a version of 1.3.0, or any Koi.Device.* path, you are on the custom pack and NOTHING else in this guide applies. Stop and change tenant. A partial count (fewer than 13) means a failed or interrupted install, not the wrong pack.

REFERENCE

All 13 commands — expected shape of each

Generated from reference/marketplace-pack.json. Totals across the pack: 13 commands, 67 arguments, 131 declared outputs.

Command	Args	Outputs	Context prefix written	Effect	Test
koi-get-events	5	4	KOI.Event.*	read-only · debug only	6
koi-policy-list	3	9	Koi.Policy.*	read-only	3
koi-policy-status-update	2	9	Koi.Policy.*	WRITE	8
koi-allowlist-get	0	9	Koi.Allowlist.*	read-only	3
koi-allowlist-items-add	5	0	none — war-room message only	WRITE	not exercised
koi-allowlist-items-remove	5	0	none — war-room message only	WRITE · execution	not exercised
koi-blocklist-get	0	9	Koi.Blocklist.*	read-only	3
koi-blocklist-items-add	5	0	none — war-room message only	WRITE	8
koi-blocklist-items-remove	5	0	none — war-room message only	WRITE · execution	8
koi-inventory-list	21	27	Koi.Inventory.*	read-only	4, 7
koi-inventory-item-get	3	27	Koi.Inventory.*	read-only	4
koi-inventory-search	7	27	Koi.Inventory.*	read-only	5
koi-inventory-item-endpoints-list	6	10	Koi.Inventory.Endpoint.*	read-only	4

Four commands declare zero outputs (koi-allowlist-items-add, koi-allowlist-items-remove, koi-blocklist-items-add, koi-blocklist-items-remove) — a war-room message only, so nothing can branch on their result. Only koi-allowlist-items-remove and koi-blocklist-items-remove carry "execution", the potentially-harmful flag; the other three WRITE commands mutate state unflagged — "not flagged" is not "safe". "Read-only" means only that KOI is unchanged: koi-get-events is debug-only and writes into koi_koi_raw when should_push_events=true, so it alone is not freely repeatable. The allowlist add/remove pair is exercised by no test here — test 8 works the blocklist pair; the sign-off slide records the same gap.

TEST 2 · GATE

Instance configuration and the Test button

Steps

1. Settings → Integrations → Instances → add an instance of KOI.
2. Set Server URL to the shipped default:

   ```
   https://api.prod.koi.security/
   ```

3. Paste the API Key. Leave insecure and proxy off unless your network needs them.
4. To collect events, tick Fetch events and keep the defaults:

   ```
   event_types_to_fetch = Alerts,Audit    max_fetch = 5000
   ```

5. Click Test, then Save. Then run a live read probe:

   ```
   !koi-policy-list limit=1
   ```

Expected result

- ✓ Test returns Success.
- ✓ The instance saves without a validation error.
- ✓ koi-policy-list returns a table and writes Koi.Policy.* to context.
- ✓ No 401 on the probe. A bad key is verified to answer 401 {"message":"Unauthorized","statusCode":401}.

! **Real failure vs empty-but-correct**

A green Test button alone is weak evidence — always follow it with one real read command. Conversely, koi-policy-list returning an empty policies list with HTTP 200 is a correct result for a tenant with no policies, not a failure. Distinguish by the error: 401 means the key (verified, VERIFIED_FACTS §5.4a); an empty table with no error means an empty tenant. No 403 was ever observed here, so if you get one, diagnose it — do not assume it proves blocked egress.

REFERENCE

Every configuration parameter, as shipped

Generated from reference/marketplace-pack.json. Use this to check an existing instance field by field.

Parameter	Display name	Section	Req.	Shipped default	Visibility
`url`	Server URL	Connect	yes	`https://api.prod.koi.security/`	visible
`api_key`	API Key	Connect	yes	—	visible
`insecure`	Trust any certificate (not secure)	Connect	no	—	visible · advanced
`proxy`	Use system proxy settings	Connect	no	—	visible · advanced
`isFetchEvents`	Fetch events	Collect	no	—	hidden on xsoar
`event_types_to_fetch`	Fetch event types	Collect	yes	`Alerts,Audit`	hidden on xsoar
`audit_types_filter`	Audit log type filter	Collect	no	—	hidden on xsoar
`max_fetch`	Maximum number of events per fetch	Collect	no	`5000`	hidden on xsoar
`eventFetchInterval`	Events Fetch Interval	Collect	no	`1`	hidden on xsoar · advanced

Two collectors writing one dataset is legal and easy to miss: if you configure a second KOI instance with Fetch events on, both write to the same dataset and the pack ships no field identifying which instance produced a row (VERIFIED_FACTS §2).

TEST 3

Read sweep — policies, allowlist, blocklist

Steps

1. In the war room, run each of the three read commands:

   ```
   !koi-policy-list limit=5
   ```

   ```
   !koi-allowlist-get
   ```

   ```
   !koi-blocklist-get
   ```

2. For each, open the context panel and check the prefix it wrote.
3. Record the policy id and enabled state of one policy — test 8 needs it.
4. Note: koi-allowlist-get and koi-blocklist-get take no arguments at all.

Expected result

- ✓ koi-policy-list writes Koi.Policy.* (9 declared fields).
- ✓ koi-allowlist-get writes Koi.Allowlist.* (9 fields).
- ✓ koi-blocklist-get writes Koi.Blocklist.* (9 fields).
- ✓ A human-readable table for each, even when the list is empty.
- ✓ No command in this test changes anything in KOI.

! **Real failure vs empty-but-correct**

Empty allowlist and blocklist are NORMAL — on KOI_PAET both were empty; on KOI_PLTS they held 5 and 17 entries (VERIFIED_FACTS §7). Do not judge these two by a total: their API responses carry only an items array and no total_count field at all, unlike policies and inventory (VERIFIED_FACTS §5.4). A guide that tells you to read total_count here is telling you to read a field that never exists.

TEST 4

Read sweep — inventory, item, endpoints

Steps

1. List a small page of inventory:

   ```
   !koi-inventory-list limit=5
   ```

2. Copy item_id, marketplace and version from one row.

3. Fetch that item — all three arguments are required:

   ```
   !koi-inventory-item-get item_id=<id> marketplace=<mp> version=<ver>
   ```

4. List the endpoints that have it installed:

   ```
   !koi-inventory-item-endpoints-list item_id=<id> marketplace=<mp>
   version=<ver> limit=5
   ```

5. Then narrow the list with a filter argument:

   ```
   !koi-inventory-list risk_level=high limit=5
   ```

Expected result

- ✓ koi-inventory-list and koi-inventory-item-get both write Koi.Inventory.* — 27 declared fields each, identical paths.
- ✓ koi-inventory-item-endpoints-list writes Koi.Inventory.Endpoint.* — 10 nested fields, no path in common with the other two.
- ✓ Endpoints are reachable ONLY from an item. There is no device-centric command and no Koi.Device.* prefix — though koi-inventory-list does take device_id.
- ✓ The risk_level DROPDOWN offers low, medium, high, critical, pending — that is the YAML's list, not a measured API contract (test 7 shows one that was incomplete).

! **Real failure vs empty-but-correct**

Three commands collide on Koi.Inventory.*: koi-inventory-list, koi-inventory-item-get and koi-inventory-search (test 5). Running any two back to back overwrites context — read it after each one, not at the end. koi-inventory-item-endpoints-list is NOT one of them. An item that lists zero endpoints WOULD be a valid result — KOI knows the item but nothing in scope currently has it installed. That is a case to be ready for, not one seen here: the one item probed on each instance returned an endpoint. A wrong item_id / marketplace / version triple is an error, not an empty list.

TEST 5

Advanced search needs a structured filter

Steps

1. Run the search with an explicit query-builder filter:

   ```
   !koi-inventory-search filter_json={"combinator":"and","rules":
   [{"field":"risk_level","operator":"=","value":"high"}]}
   ```

2. Record its total_count, then run the nearest plain-argument form:

   ```
   !koi-inventory-list risk_level=high limit=5
   ```

3. Run it with NO filter argument, then again with a malformed one:

   ```
   !koi-inventory-search
   ```

   ```
   !koi-inventory-search filter_json={}
   ```

4. Optionally supply the filter as a war-room JSON file instead:

   ```
   !koi-inventory-search filter_raw_json_entry_id=<entry id>
   ```

Expected result

- ✓ The filtered search writes Koi.Inventory.* (27 fields).
- ✓ Both return a count. Whether they agree was never measured — record both, assert nothing.
- ✓ No filter argument: the INTEGRATION refuses before any API call — "Either 'filter_json' or 'filter_raw_json_entry_id' must be provided."
- ✓ filter_json={}: HTTP 400, and the body names the bad keys (filter.combinator, filter.rules).
- ✓ YAML documents filter_raw_json_entry_id as taking priority over filter_json — never exercised; verify it.

! **Real failure vs empty-but-correct**

Three distinct failures — do not conflate them (VERIFIED_FACTS §5.3). No filter argument: the integration stops it, no API call happens — that one is read from Koi.py, not reproduced. Malformed filter: HTTP 400 whose body names the problem ("filter.combinator must be one of the following values: and, or", "filter.rules must be an array") — observed on both instances, not bare or unexplained. HTTP 500 is reachable only by calling the API directly with the filter key omitted, never through the command. Zero results from a valid filter is a correct answer.

TEST 6

koi-get-events — debugging only, push disabled

Steps

1. Run it with a small limit and pushing explicitly off:

   ```
   !koi-get-events limit=5 should_push_events=false
   ```

2. Read the returned table and the context it writes.

3. Optionally scope it to one type and a time window:

   ```
   !koi-get-events event_type=Alerts start_time="3 days ago"
   should_push_events=false
   ```

4. Do NOT leave should_push_events=true in any repeated or scheduled use.

Expected result

- ✓ Writes KOI.Event.* — note the upper-case prefix, it differs from every other command.
- ✓ event_type accepts Alerts and Audit; the default is Alerts,Audit.
- ✓ should_push_events defaults to false, which is what you want here.
- ✓ Events appear in the war room without being written to the dataset.

! **Real failure vs empty-but-correct**

The pack's own description calls this command development-and-debugging only: it may produce duplicate events, exceed API rate limits, or disrupt the fetch mechanism. So unlike tests 3, 4, 5 and 7 it is NOT safe to repeat freely — run it once, deliberately, with should_push_events=false; setting it true writes events into koi_koi_raw. Use it to prove the API returns events — never as a substitute for the collector in test 9. Returning no events for a narrow window on a quiet tenant is correct.

TEST 7 · KNOWN DEFECT

The mcp_servers view works but is not in the dropdown

Steps

1. Open the view argument's dropdown on koi-inventory-list.
2. Count the offered values — there are 6:

   `agentic_ai, ai_models, code_packages, extensions, os_packages, software`
3. Now type the value by hand instead of picking it:

   `!koi-inventory-list view=mcp_servers limit=5`
4. Repeat for the other two values the dropdown omits:

   `view=all_items` `view=repositories`
5. Finally, confirm the shipped example value is broken:

   `!koi-inventory-list view=browser_extensions`

Expected result

- ✓ The dropdown offers 6 values; the API accepts 9.
- ✓ Missing from the dropdown but accepted: all_items, mcp_servers, repositories.
- ✓ All 12 were probed on BOTH instances (§5.1): mcp_servers and repositories return 200 and a non-zero count.
- ✓ all_items is ACCEPTED (200) but returned total_count 0 on BOTH tenants — a zero, not a rejection.
- ✓ browser_extensions returns 400, as do ide_extensions and packages — all three are in neither list.

! **Real failure vs empty-but-correct**

Both halves of this test are expected results, not failures. The YAML's predefined list is incomplete, not wrong: nothing it offers is invalid. An empty result for view=mcp_servers means your estate has no MCP servers in scope; view=all_items returned zero rows on both tenants probed even though the API accepted it, so prefer omitting view entirely — that returned the whole inventory: 3,447 on KOI_PAET, 5,644 then 5,646 hours later on KOI_PLTS. Only KOI_PLTS moved between the two sweeps; name the instance whenever you quote a count, and read either as scale, never as a target (§5.1, §7). Only a 400 — a rejected value — is a failure.

TEST 8 · STATE-CHANGING

The five commands that write to KOI — controlled protocol

Do not fire these blind. Each one changes governance state in KOI, not in Cortex, so an undo is your responsibility. Four of them declare no outputs at all, which means the war-room message is your only record of what happened.

The five, from the pack JSON

`koi-policy-status-update`	9 outputs
`koi-allowlist-items-add`	0 outputs
`koi-allowlist-items-remove`	execution · 0 outputs
`koi-blocklist-items-add`	0 outputs
`koi-blocklist-items-remove`	execution · 0 outputs

Real failure vs empty-but-correct

The four add/remove commands declare zero outputs, so success is a war-room message and nothing else — an empty context is CORRECT. The only proof either way is a follow-up -get. Note too that only the two removes carry execution: true — the two adds and the policy toggle mutate governance state with no flag at all, so "not flagged" is not "safe".

Protocol — record, change, verify, reverse · BLOCKLIST pair only

1. Pick a DISPOSABLE test item you are willing to have on a governance list.
2. Record the before state — run koi-blocklist-get and export the result.

```
!koi-blocklist-items-add item_id=<id> marketplace=<mp>
notes="test"
```

3. Re-run koi-blocklist-get: the item now appears.

```
!koi-blocklist-items-remove item_id=<id> marketplace=<mp>
```

4. Re-run koi-blocklist-get a third time — it matches the before state.

koi-policy-status-update — the same discipline, plus one difference

Both arguments are required (policy_id, enabled; enabled accepts true / false). Unlike the four list writes it DOES return context — 9 fields under Koi.Policy.*, the same paths koi-policy-list writes, so the toggle overwrites the list you captured in test 3. Take a policy's enabled value from test 3, flip it, verify with koi-policy-list, then flip it back and verify again. Never toggle a policy that is actively enforcing on a production estate. None of these five commands was run while preparing this guide (VERIFIED_FACTS §1.1) — treat this slide as a protocol to follow, not as observed behaviour, and do not assume a repeated add or a redundant remove is a harmless no-op until you have seen it.

BEFORE TESTS 9-10

How to make koi_koi_raw actually move

Tests 9 and 10 read a dataset that only fills under specific conditions. Get these three wrong and a healthy pack looks broken. All three were established on a live tenant on 21 July 2026.

1

KOI is run-on-demand — trigger a scan, and change something

KOI has no resident Windows agent. koi_koi_raw updates only after a scan runs — core-script-run of "KOI Deployment Script - Windows" (COMPLETED_SUCCESSFULLY in ~135s). Both test hosts went six days with no events until a scan ran — nothing was wrong with them. And it is change-driven, not scan-driven: a scan of an UNCHANGED host produced no events at all. A quiet host returning an empty dataset is correct, not a failure (§7b, §8).

2

Detection follows the PATH, not the installing process identity

A change is inventoried only if it lands in a USER-profile path; a SYSTEM-profile install is invisible. A controlled three-way test settled it: tabulate 0.9.0 written to the SYSTEM profile → 404, not detected; inflection 0.5.1 and a git clone into the user profile → both detected within minutes. All three were driven by the SYSTEM Cortex agent, so the path decides. Detection CAN be automated through the EDR agent — the change need only land in a user profile, on an item verified absent first (a re-install of an already-inventoried item is a no-op) (§7d.1, §7d.2).

3

Events and inventory are separate surfaces with different latency

The event stream updates within minutes of a scan (~4–10 min end to end). The inventory API index LAGS: an item present in events at 06:06 was still absent from GET /inventory at 06:21 — total_count 0, and the item endpoint 404s. Treat an empty inventory lookup as INCONCLUSIVE, never as proof the item is absent; react-to-event-then-enrich flows must run these lookups continue-on-error and must not assert absence from an empty result (§7d.3).

TEST 9

Event collection lands in koi_koi_raw

Steps

1. Precondition: a scan ran and something changed on the host (preceding slide).
2. Confirm Fetch events is enabled and save — XSIAM / platform only.
3. Wait for two fetch cycles (default 1 min), then count each stream:

```
dataset = koi_koi_raw | filter source_log_type="Audit" | comp
count() as audit
```

```
dataset = koi_koi_raw | filter source_log_type="Alerts"
| comp count_distinct(json_extract_scalar(metadata,
      "$.notification_event_id")) as alerts
```

4. Re-run after one more cycle; then confirm the tags:

```
dataset = koi_koi_raw | comp count() as n by _vendor, _product
```

Expected result

- ✓ The dataset koi_koi_raw exists and returns rows.
- ✓ One row per source_log_type — Alerts and Audit.
- ✓ Distinct alert count (not row count) reflects real activity — Alert ROWS re-inflate every fetch (~245×/24h, §7e); Audit is not duplicated.
- ✓ _vendor = koi and _product = koi.

Real failure vs empty-but-correct

The dataset name is not declared anywhere in the pack — it follows the {vendor}_{product}_raw convention and was confirmed on a live tenant (VERIFIED_FACTS §3). koi_koi_raw only moves after a scan runs and something changes (§7b/§8). XSIAM / platform only: the YAML sets isfetchevents true then overrides it with isfetchevents:xsoar false, so on XSOAR there is no collector. "Dataset not found" means no event has EVER arrived. Never count Alert ROWS between cycles — they grow on every fetch regardless of new activity; count_distinct on metadata.notification_event_id instead (§7e).

TEST 10

Alerts vs Audit — and why XDM is empty

Steps

1. Separate the two schemas — source_log_type is the discriminator:

```
dataset = koi_koi_raw | filter source_log_type = "Audit"
| comp count() as n by type, category
```

```
dataset = koi_koi_raw | filter source_log_type = "Alerts"
| fields _time, class_uid, type_uid, severity_id, status_id
```

2. Now check the data model — expect nothing:

```
dataset = koi_koi_raw | fields xdm.*
```

3. And confirm the alert_type trap:

```
dataset = koi_koi_raw | filter alert_type != null | comp count()
```

Expected result

- ✓ Audit rows are flat and KOI-native: type, action, category, object_name, object_type, hostname, item_version.
- ✓ Alert rows are OCSF: class_uid 2007, type_uid 200701, is_alert true, plus severity_id / confidence_id / risk_level_id / status_id.
- ✓ Every xdm.* field is EMPTY. This is correct — the pack ships no parsing rule and no modeling rule.
- ✓ alert_type returns zero rows. It is never populated.

Real failure vs empty-but-correct

Empty XDM is the expected result here, not a failure — there is nothing in this pack to populate it, so do not raise a bug and do not build queries on xdm.*. Two more traps: on alert rows, resources, observables and metadata are JSON STRINGS and must be json_extract-ed before you can filter on them — a query that treats them as structured fields silently returns nothing rather than erroring; and Audit fields are null on Alert rows and vice versa, so always filter by source_log_type first.

MOST CONSEQUENTIAL FINDING

Alerts are duplicated on every fetch — dedupe every count

The integration re-sends every still-open alert on each 1-minute fetch, so koi_koi_raw holds one row per alert PER FETCH, not one per alert. Audit is unaffected — each record carries a unique KOI id. Measured 21 July 2026 (§7e).

Rows vs distinct alerts

Stream	Window	Rows	Distinct	Inflation
Alerts	last 24 h	734	3	~245×
Alerts	last 90 d	1,048	317	3.3×
Audit	last 24 h	257	257	1.0
Audit	last 90 d	20,148	20,148	1.0

The only correct dedupe key (90 d, 1,048 rows)

`metadata.notification_event_id` — 317 distinct
the alert occurrence ✅

`_id` — 1,048 distinct
the row — every duplicate

`observables[event.id]` (koi_event_id) — 20 distinct
the scan batch — too coarse

`finding_info.uid` (finding_uid) — 3 distinct
the finding/policy definition

Every count() over Alerts is wrong — count_distinct on the notification id

```
dataset = koi_koi_raw | filter source_log_type="Alerts"
| comp count_distinct(json_extract_scalar(metadata,"$.notification_event_id")) as alerts
```

Within one notification: 357 rows, 1 distinct _time, 1 message, 357 distinct _insert_time — identical content re-inserted every cycle. notification_event_id is a verified 1:1 identity for (item.id, device.id, finding_info.uid, finding_info.created_time). Historical rows carry no promoted column, so dedupe inline. Figures drift as the dataset grows — read them as ratios, never as target counts (§7e).

JUDGING RESULTS

Empty-but-correct, or a real failure?

Most false alarms on this pack come from reading an empty-but-valid result as a fault. One row per test.

Test	What you might see	Empty but correct when…	A real failure when…
1	Fewer than 13 commands	never — this test is pass/fail	any count other than 13, or version ≠ 1.2.3
2	Test button green, command errors	n/a — always follow Test with a read	401 {"message":"Unauthorized"} = the key (verified). Any other status: diagnose it, do not assume egress
3	Allowlist or blocklist has no rows	the tenant has no entries; both were empty on KOI_PAET	an HTTP error, or you were told to read total_count (it does not exist here)
4	An item lists zero endpoints	the item is not currently installed in scope	wrong item_id / marketplace / version triple
5	koi-inventory-search returns nothing	the filter is valid and matches nothing	HTTP 400 naming filter.combinator / filter.rules — the shape, not the data
6	koi-get-events returns no events	quiet tenant or narrow window	auth or rate-limit error
7	view=mcp_servers returns no items	no MCP servers in your estate	HTTP 400 — the value was rejected
8	add / remove writes no context	always — all four declare zero outputs (test 8 runs the blocklist pair)	the follow-up -get does not reflect the change
9	Alert ROW count grows every cycle	always — Alerts re-send each fetch; count_distinct(notification_event_id) instead (§7e)	"dataset not found", or distinct alert/Audit counts flat after a real scanned change
10	xdm.* is entirely empty	always — no modeling rules ship with this pack	raw fields empty too, on rows that exist

REFERENCE VALUES

Measured on real tenants — for shape, not as targets

Recorded 20 July 2026 on tenant api-ayman.xdr.eu.paloaltonetworks.com, instances KOI_PAET and KOI_PLTS (VERIFIED_FACTS §7). These are not pass/fail thresholds, and no test here passes by matching a number. Counts drift per tenant and between runs: where two sweeps hours apart differed — only KOI_PLTS did — both are shown, earlier then later. Quote the instance whenever you quote a count.

Measure	KOI_PAET	KOI_PLTS	Which test
Inventory items	3,447	5,644 then 5,646	4
Policies	32	28	3
Allowlist entries	0 — empty	5	3
Blocklist entries	0 — empty	17	3, 8
view=mcp_servers	21	42	7
view=software	406	1,044 then 1,046	7
Alerts available via API	296	48,526	6
Audit records available via API	8,789	96,196	6
filter risk_level = high (search)	145	not verified	5

! Scope of the evidence: measured against the KOI API directly, with the same bearer auth and API keys the two instances use. Only 8 of the 13 commands had their API exercised — the 5 state-changing ones were deliberately not run (VERIFIED_FACTS §1.1) — and none of the 13 was executed through XSIAM on that tenant (§6), so context mapping and human-readable output on these slides are asserted from the pack YAML, not observed. Running this guide from a war room is exactly what closes that gap.

OUT OF SCOPE

Tests that do NOT apply to this pack, and why

These exist in the custom pack's test guide. Every one of them depends on something this pack does not contain. Do not run them, and do not report them as failures.

Dropped test	Why it cannot run here
Alert triage, end to end	The pack ships no playbooks at all — and triage needs koi-koidex-risk-report, which does not exist here.
Item investigation	Needs koi-approval-requests-list, koi-koidex-risk-report and koi-remediations-list — none exist.
Device investigation	Needs koi-device-inventory-get and koi-remediations-list. No device-centric command and no Koi.Device.* — but koi-inventory-list takes device_id.
Gated response / approval	Needs the response playbook. The blocklist write here is a bare command with no approval step around it.
Dashboard and data model	No dashboard ships, and no modeling rules — every xdm.* field stays empty (test 10).
Collector cursor inspection	Needs koi-fetch-context-get / -set. Fetch state cannot be inspected from a command.
Egress probe via user listing	Needs koi-users-list. Use koi-policy-list limit=1 as the probe instead (test 2).

The 13 commands that do NOT exist in this pack: koi-devices-list · koi-device-inventory-get · koi-koidex-risk-report · koi-koidex-search · koi-remediations-list · koi-approval-requests-list · koi-findings-list · koi-users-list · koi-groups-list · koi-runtime-policies-list · koi-runtime-policy-get · koi-fetch-context-get · koi-fetch-context-set

ADDITIONAL CONTENT

Script Runner playbooks — portable, but not in this pack

! NOT PART OF THE MARKETPLACE KOI PACK v1.2.3. This pack ships an integration and nothing else — no playbooks. The three playbooks below come from the separate custom pack. If you deploy them you are adding content, and you must say so on any test report.

They are worth knowing about because they are the one workflow that transfers unchanged: they call no koi-* command at all, only Cortex-native ones. Nothing in them depends on which KOI pack is installed — or on any KOI pack being installed.

Koi Unified - Script Runner

The Job entry point

Koi Unified - Process Config Entry

Handles one configuration entry

Koi Unified - Execute Endpoint Script

Dispatches the script to endpoints

The only KOI commands they invoke: none

`core-get-scripts` `core-get-endpoints` `core-script-run`

Those three are Cortex-native — but the playbooks are not command-free: they also use Print, PrintErrorEntry, SetAndHandleEmpty, DeleteContext, GetErrorsFromEntry and the builtin closeInvestigation. None of it touches KOI: no KOI API key, no KOI integration instance, no koi- command.*

If you test them, test them separately

Import the three playbooks and their JSON configuration List by hand, attach the entry-point playbook to a time-triggered Job, and record the result under a heading that names them as additional content. Their prerequisites are Cortex agents and a script package in the Scripts Library — not anything from KOI. Keep the result out of the pack's own pass/fail matrix: a Script Runner failure says nothing about the Marketplace KOI pack v1.2.3, and a Script Runner pass proves nothing about it either.

SIGN-OFF

One line of evidence per test

1	Pack version reads 1.2.3 and exactly 13 koi-* commands are registered
2	Test returns Success and koi-policy-list limit=1 returns without a 401
3	koi-policy-list / -allowlist-get / -blocklist-get write Koi.Policy.*, Koi.Allowlist.*, Koi.Blocklist.*
4	Inventory commands write Koi.Inventory.* and Koi.Inventory.Endpoint.*
5	koi-inventory-search parses a query-builder filter; a malformed one 400s naming the bad key; no filter is refused by the integration
6	koi-get-events writes KOI.Event.* with should_push_events=false
7	view=mcp_servers returns 200 when typed by hand; view=browser_extensions 400s
8	Blocklist add appears and remove reverses it to the before state; a policy's enabled value flips and is put back
9	koi_koi_raw returns rows split by source_log_type, _vendor=koi; Alert counts deduped on notification_event_id (§7e)
10	Audit and Alert schemas confirmed distinct; every xdm.* field empty, as expected

All ten green → the Marketplace KOI pack v1.2.3 is installed, configured, collecting, its 8 non-mutating commands exercised, and 3 of its 5 governance writes run in test 8 — the blocklist add-and-reverse pair, plus koi-policy-status-update flipped and put back. The other 2, the allowlist pair, stay untested by these ten checks.

Record on the report: which pack and version, which tenant and instance, the date, and — for any count you quote — the instance it was measured on. Two tenants running the same pack differ by more than an order of magnitude in event volume, so a bare number means nothing without its tenant.